

Carlos Garcia

Reto:day11

<https://github.com/carlos3474004/30-dia-de-codigo/tree/main/hackerrank/day11>

The screenshot shows a HackerRank challenge completion interface. At the top, a dark blue header contains a badge for '30 Days of Code' with two stars, a message 'You have earned 30.00 points!', and a progress bar at 63% (12/15 challenges). Below this is a green banner with 'Congratulations' and a 'Next Challenge' button. A sidebar on the left lists test cases 0 through 5, all marked as completed. The main area shows a 'Compiler Message' of 'Success' and an 'Input (stdin)' section with a 'Download' link. The input data is as follows:

| Line | Input       |
|------|-------------|
| 1    | 1 1 1 0 0 0 |
| 2    | 0 1 0 0 0 0 |
| 3    | 1 1 1 0 0 0 |
| 4    | 0 0 2 4 4 0 |
| 5    | 0 0 0 2 0 0 |
| 6    | 0 0 1 2 4 0 |

¡Llegamos al **Día 11 de "30 Days of Code"** en HackerRank! Este reto es fantástico para trabajar con **matrices bidimensionales (arreglos 2D)**.



¿De qué trata el reto?

Nos dan una matriz de  $6 \times 6$  y debemos buscar una figura llamada **reloj de arena (hourglass)**, formada por 7 elementos en la siguiente forma de "I" mayúscula:

Plaintext

Un reloj de arena ocupa 3 filas de alto y 3 columnas de ancho. En una matriz de  $6 \times 6$ , se pueden formar exactamente **16 relojes de arena**. Debemos calcular la suma de cada uno y devolver la **suma máxima**.



**La trampa del ejercicio**

Los valores en la matriz pueden ser **negativos** (entre  $-9$  y  $9$ ).

Si inicializas la variable de la suma máxima en `0`, fallarás los casos donde todas las sumas de los relojes de arena sean negativas (por ejemplo, si la suma máxima da `-6, 0` sería incorrecto). Debemos inicializar el valor máximo con un valor muy pequeño (como `-infinity` o la menor suma posible, que es  $7 \times (-9) = -63$ ).



## Código de la Solución (Limpio para HackerRank)

Puedes reemplazar el contenido del editor en HackerRank por este código:

### Python

```
import sys

def main():
    # Read all lines from standard input
    lines = sys.stdin.readlines()

    if not lines:
        return

    # Build the 6x6 2D matrix
    matrix = []
    for line in lines:
        if line.strip():
            row = list(map(int, line.split()))
            matrix.append(row)

    # Initialize maximum sum to a very small number
    max_sum = -64

    # Iterate through possible top-left corners of each hourglass
    # Rows 0 to 3, Columns 0 to 3
    for r in range(4):
        for c in range(4):
            # Calculate the current hourglass sum
            sum = matrix[r][c] + matrix[r][c+1] + matrix[r][c+2] + \
                  matrix[r+1][c-1] + matrix[r+1][c] + matrix[r+1][c+1] + \
                  matrix[r+2][c-2] + matrix[r+2][c-1] + matrix[r+2][c] + \
                  matrix[r+2][c+1] + matrix[r+2][c+2]

            # Keep track of the maximum sum found
            if sum > max_sum:
                max_sum = sum

    print(max_sum)

if __name__ == '__main__':
    main()
```



¿Por qué los rangos son `range(4)`?

El patrón de 3x3 necesita espacio para extenderse desde la posición superior izquierda `(r, c)`:

- Fila superior:  $r$
- Fila media:  $r + 1$
- Fila inferior:  $r + 2$

Si  $r$  llegara a ser  $4$  o  $5$ , al intentar acceder a  $r + 2$  estaríamos fuera del límite de la matriz ( $6$  filas total, índices  $0$  a  $5$ ). Por tanto, los índices válidos para la esquina superior izquierda van de  $0$  a  $3$ .